COS 214 Practical 4
Group 47

Azolile Mbanga, Chavonne Makurira, Kayla Falconer

System Description:

The system represents the structure of a veterinary hospital. A hospital consists of departments, which contain rooms, and then individual patients.

Composite Pattern is used for specifying the areas in the veterinary system where patients are situated. **Vet** is the common base class, **Unit** represents containers such as Emergency Department, Treatment Room, Surgery Department, Treatment Room Theatre, and Ward, and **Patient** represents the leaf objects. Because a **Unit** can contain other **Vet** objects, a hierarchy of any depth can be constructed.

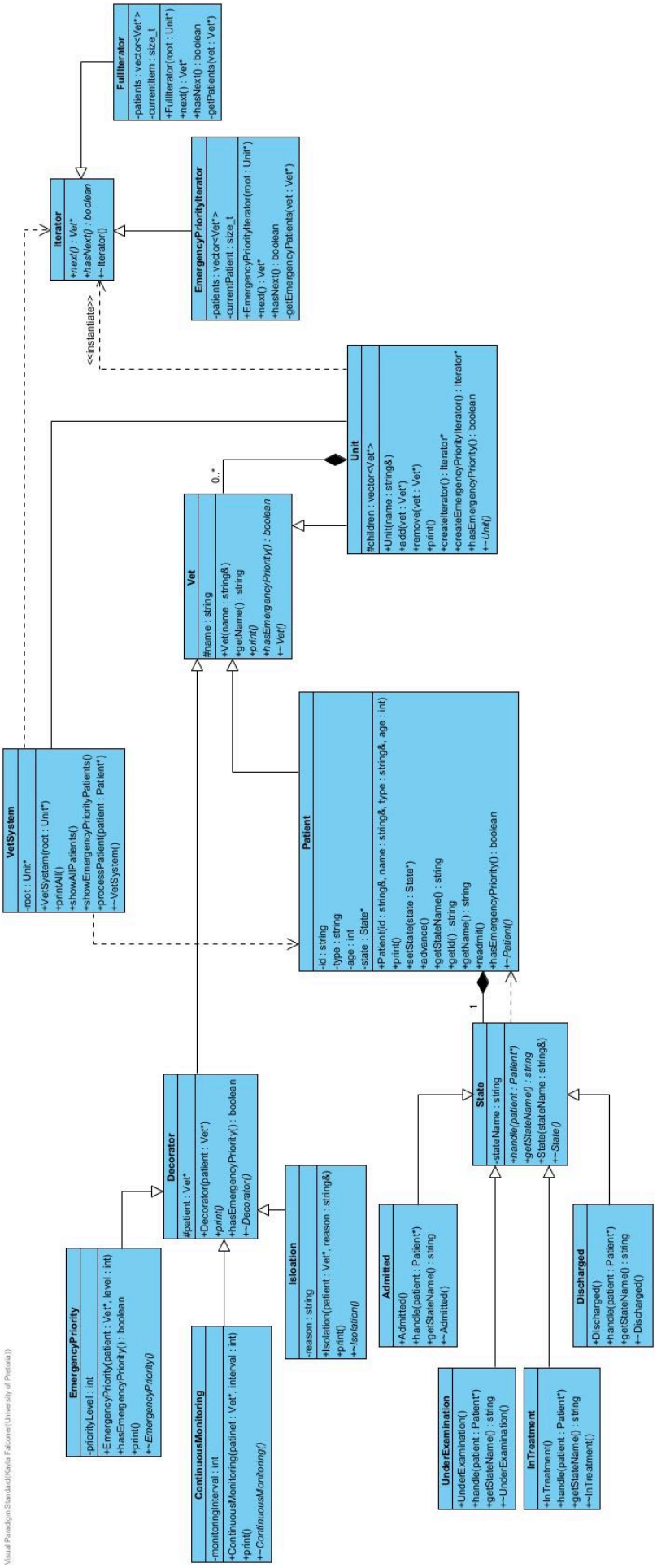
The **Iterator Pattern** allows for moving through the veterinary system hierarchy without exposing the Unit's internal vector. We have a **FullPatientIterator** for finding all patients and an **EmergencyPriorityIterator** for finding only emergency-priority patients. This gives us two different traversal behaviours over the same hierarchy.

The **State Pattern** manages where a patient is in their treatment process. The patient's current **State** determines which actions are valid. For example, treating a patient who is still only **Admitted** can be rejected, while treating an **UnderExamination** patient is valid. This a large if or switch statement is not necessary to manage the patient's lifecycle.

The **Decorator Pattern** allows for the dynamic adding of special characteristics to patients without creating lots of different Patient subclasses. For example, a patient can be wrapped with **EmergencyPriorityDecorator**, **IsolationDecorator**, or **ContinuousMonitoringDecorator**. Decorators can also be stacked, so one patient could simultaneously be emergency priority, isolated, and continuously monitored.

VetSystem is the main **client**. It ties everything together and holds the root **Unit**, uses iterators to find patients, and can process patients through their states.

UML Class Diagram:



GoF Participant Mapping:

Composite Pattern:
Client – VetSystem
Component – Vet
Composite – Unit
Leaf – Patient

Decorator Pattern:
Client – VetSystem
Component – Vet
ConcreteComponent – Patient
Decorator – Decorator
ConcreteDecorator - Isolation, ContinuousMonitoring, EmergencyPriority

State Pattern:
Client – VetSystem
Context – Patient
State – State
ConcreteState - Admitted, UnderExamination, InTreatment, Discharged

Iterator Pattern:
Client - VetSystem
Aggregate - Unit
ConcreteAggregate - Unit
Iterator - Iterator
ConcreteIterator - FullIterator, EmergencyPriorityIterator

Concise Design/Ownership Rationale:

- Vet is the component (common abstraction) for both nested Unit objects and individual Patient objects, meaning a veterinary system hierarchy can be treated uniformly using the Composite pattern
- Vet has **generalization** (is-a) between both Unit and Patient – they inherit from Vet
- Vet also makes use of **composition** (owns-a) for its relationship to Unit – a Unit cannot exist without a Vet component
- A Unit component owns its Patient component through its vector attribute

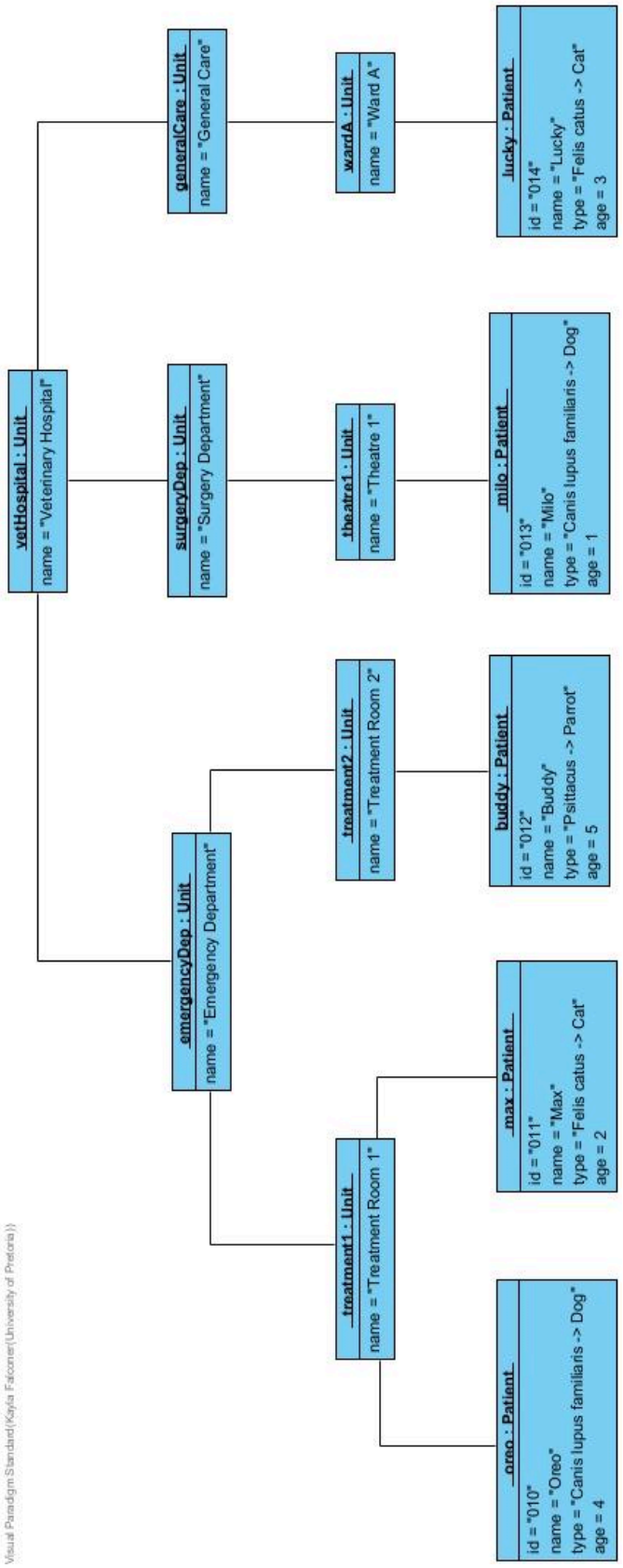
- Patient makes use of **composition** (owns-a) for its relationship to State, and therefore Patient is responsible for instantiating and deleting its State object when its internal state changes

- Vet has **generalization** (is-a) with Decorator, and Decorator stores the Vet component it is decorating

- Iterator ensures that it does not have access to the internal state of Vet through its lack of inheritance

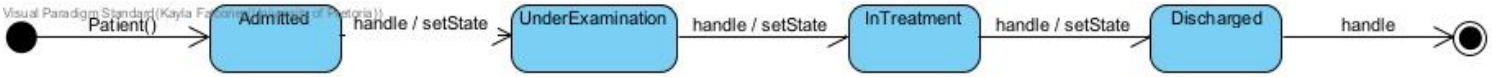
UML Object Diagram:

Showing runtime portion of nested hierarchy



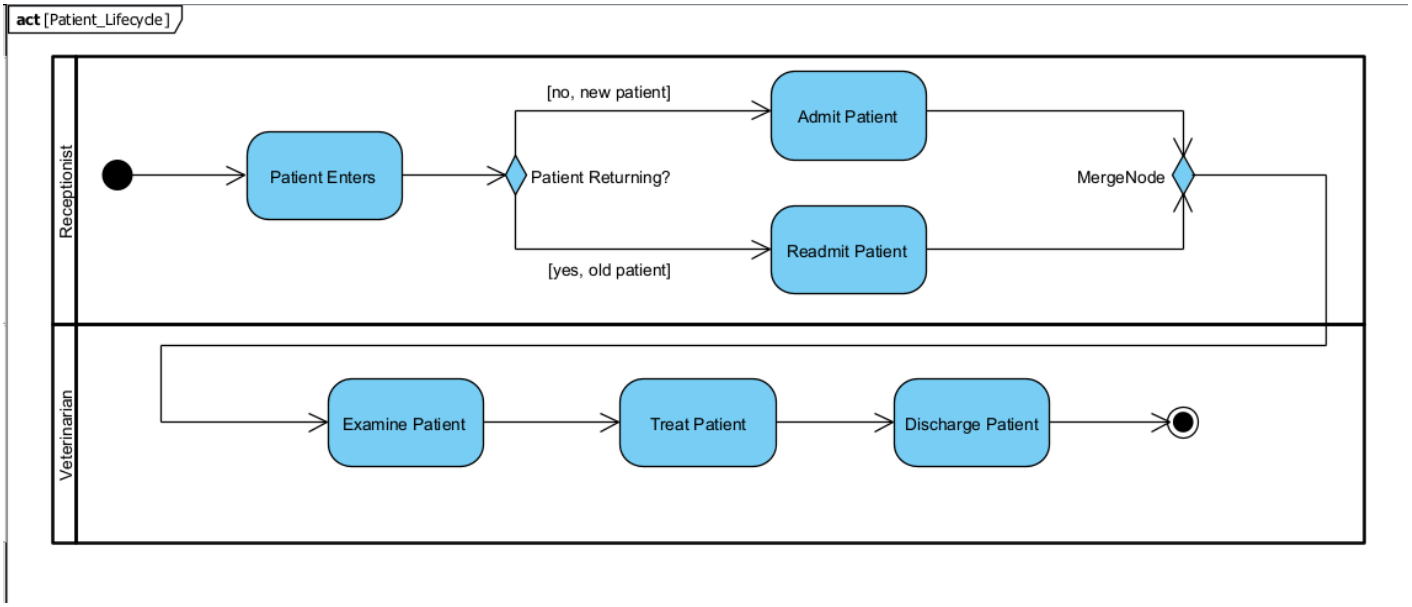
UML State Diagram:

Showing lifecycle of stateful object

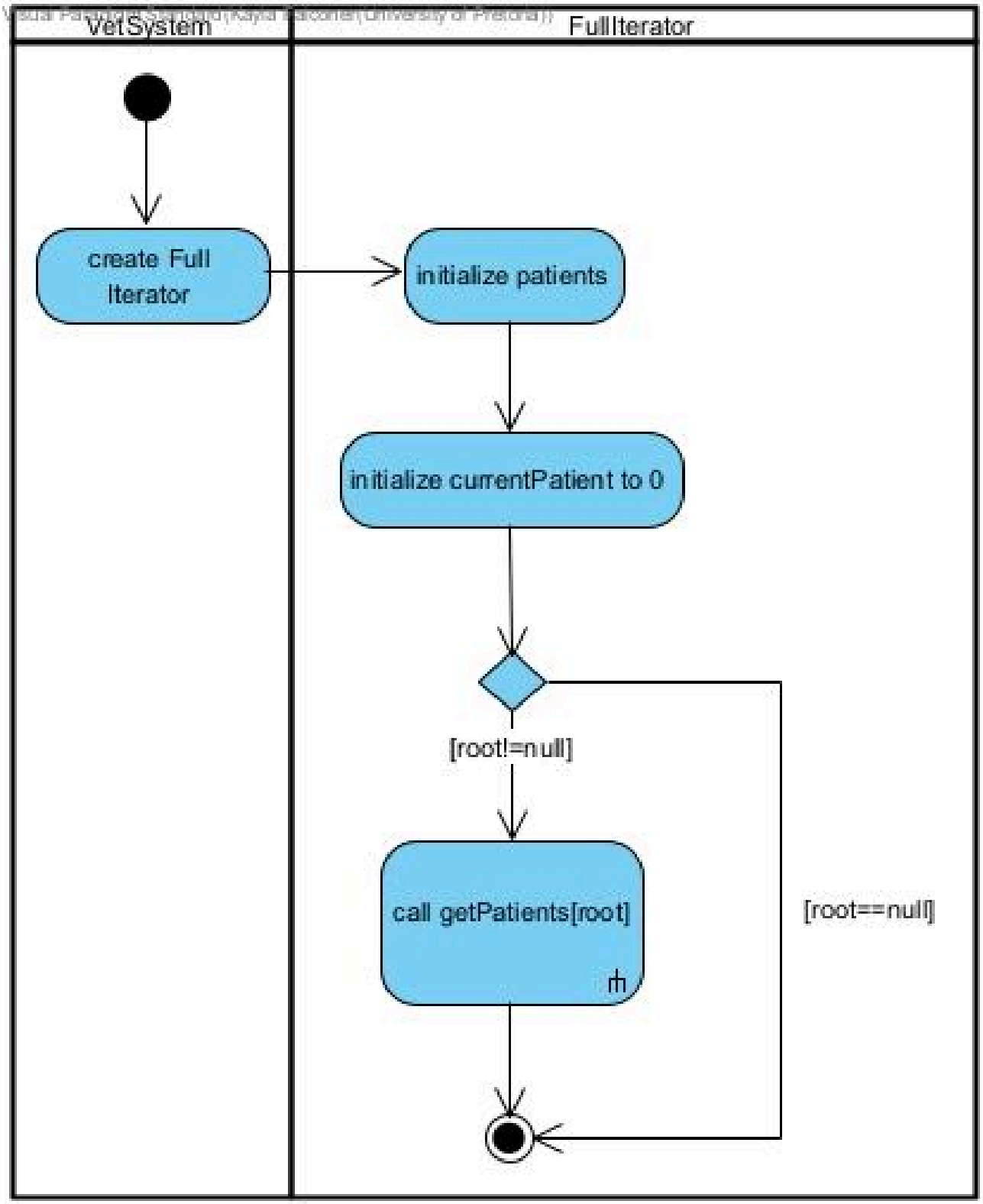


UML Activity Diagrams:

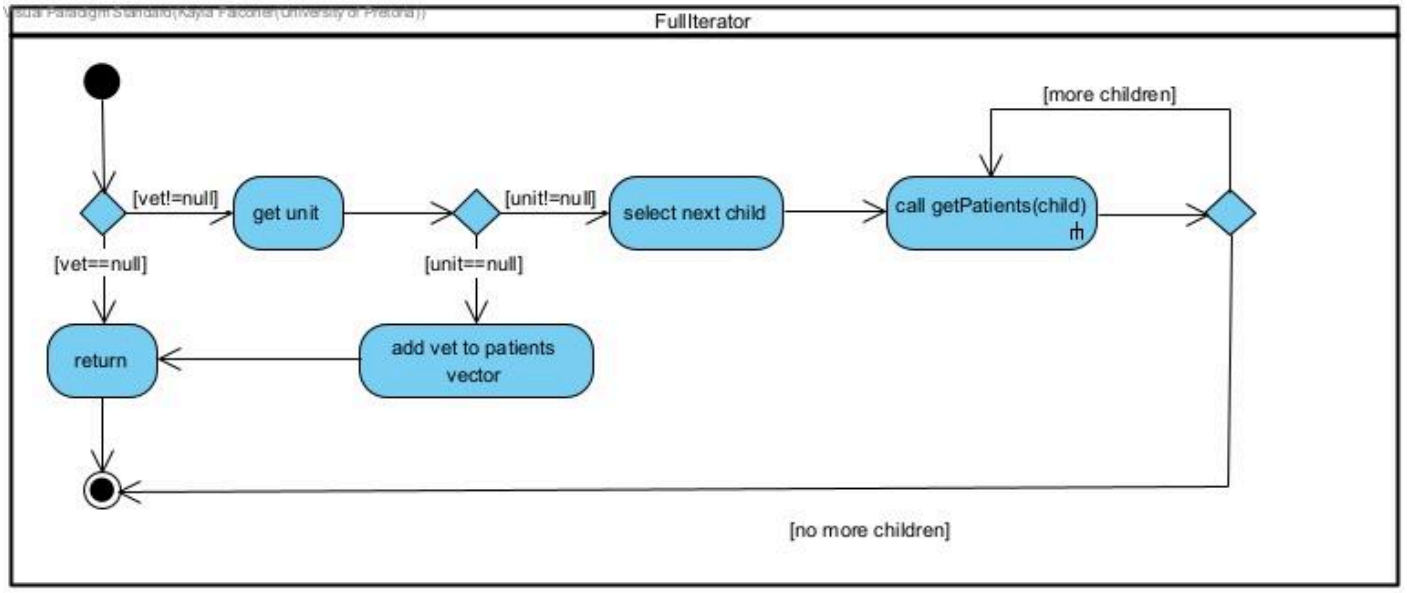
Showing Patient Lifecycle



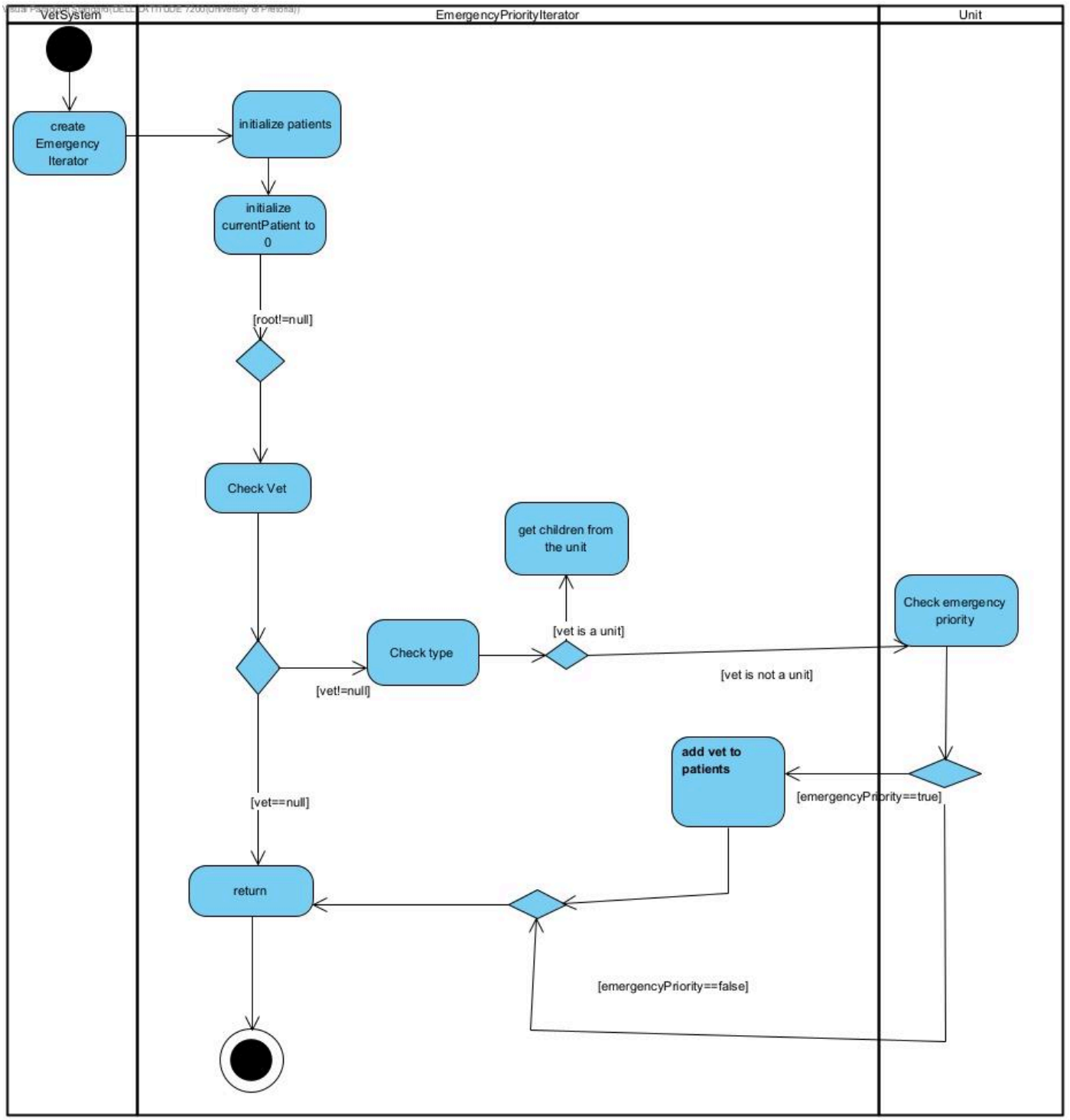
Showing FullIterator



Expanding Composite in FullIterator



Showing EmergencyPatientIterator



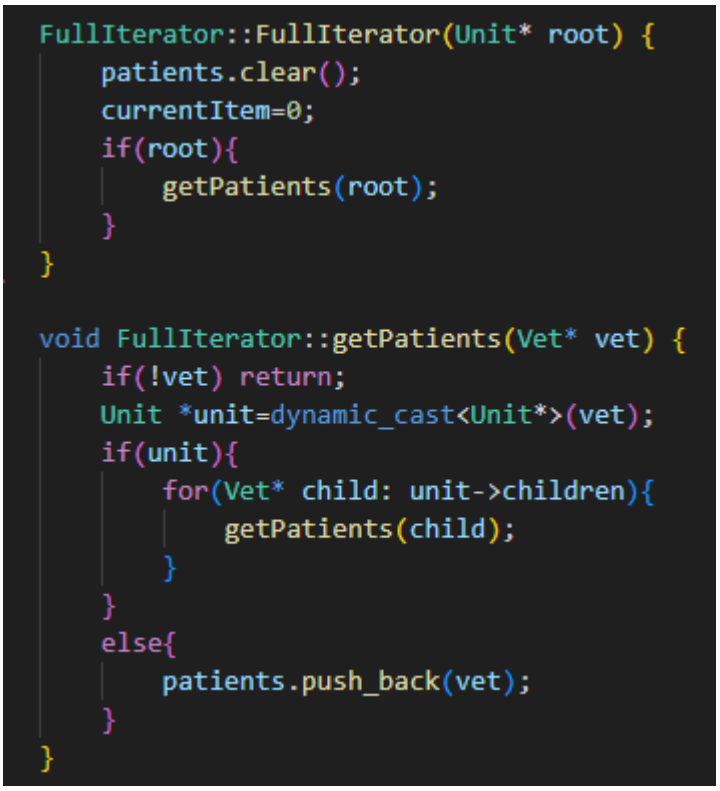
Traversal-Modification Policy:

FullIterator and EmergencyPriorityIterator both make use of snapshots of the Patient pointer variables at the time the iterator is created.

This means that if a change is made, such as adding or removing a Patient from the Vet hierarchy, after the iterator has been created, it will not reflect.

A new iterator would have to be implemented to include this change.

To avoid dangling pointers, Patient objects must not be removed during the traversal sequence of the iterator. One may only be deleted once the iterator traversal has fully completed.



```
FullIterator::FullIterator(Unit* root) {
    patients.clear();
    currentItem=0;
    if(root){
        getPatients(root);
    }
}

void FullIterator::getPatients(Vet* vet) {
    if(!vet) return;
    Unit *unit=dynamic_cast<Unit*>(vet);
    if(unit){
        for(Vet* child: unit->children){
            getPatients(child);
        }
    }
    else{
        patients.push_back(vet);
    }
}
```

GDB Bug investigation:

A bug we encountered had a symptom of a segmentation fault at state->getName() in the Patient::setState(State* state) function. The cause was because readmit() called setState(nullptr). During the debugging process using GDB print state showed 0x0(null). The fix was to change setState(nullptr) to setState(new Admitted()) in the readmit() function. Other additional fixes was to add null checks to the print function and setState() function in Patient.cpp

Segfault proof:

```
DELL LATITUDE 7200@CV-PC MINGW64 ~/Documents/COS214_prac4 (Chavonne)
$ docker run --rm taskforge-image
Patient Buggy state changed to Under Examination
Patient Buggy state changed to In Treatment
Patient Buggy state changed to Discharged

DELL LATITUDE 7200@CV-PC MINGW64 ~/Documents/COS214_prac4 (Chavonne)
$ docker run --rm taskforge-image valgrind --leak-check=full ./taskforge
==1== Memcheck, a memory error detector
==1== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1== Command: ./taskforge
==1==
Patient Buggy state changed to Under Examination
Patient Buggy state changed to In Treatment
Patient Buggy state changed to Discharged
==1== Invalid read of size 8
==1==    at 0x10CEA8: Patient::setState(State*) (Patient.cpp:30)
==1==    by 0x10D0AD: Patient::readmit() (Patient.cpp:49)
==1==    by 0x10E447: main (main.cpp:18)
==1==  Address 0x0 is not stack'd, malloc'd or (recently) free'd
==1==
==1==
==1== Process terminating with default action of signal 11 (SIGSEGV)
==1==  Access not within mapped region at address 0x0
==1==    at 0x10CEA8: Patient::setState(State*) (Patient.cpp:30)
==1==    by 0x10D0AD: Patient::readmit() (Patient.cpp:49)
==1==    by 0x10E447: main (main.cpp:18)
==1==  If you believe this happened as a result of a stack
==1==  overflow in your program's main thread (unlikely but
==1==  possible), you can try to increase the size of the
==1==  main thread stack using the --main-stacksize= flag.
==1==  The main thread stack size used in this run was 8388608.
==1==
==1== HEAP SUMMARY:
==1==     in use at exit: 76,960 bytes in 4 blocks
==1==   total heap usage: 10 allocs, 6 frees, 77,134 bytes allocated
==1==
==1== 40 bytes in 1 blocks are definitely lost in loss record 1 of 4
==1==    at 0x4849013: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1==    by 0x10C826: InTreatment::handle(Patient*) (InTreatment.cpp:14)
==1==    by 0x10CE06: Patient::advance() (Patient.cpp:21)
==1==    by 0x10E438: main (main.cpp:17)
==1==
==1== LEAK SUMMARY:
==1==    definitely lost: 40 bytes in 1 blocks
==1==    indirectly lost: 0 bytes in 0 blocks
==1==      possibly lost: 0 bytes in 0 blocks
==1==    still reachable: 76,920 bytes in 3 blocks
==1==         suppressed: 0 bytes in 0 blocks
==1== Reachable blocks (those to which a pointer was found) are not shown.
==1== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)

valgrind: the 'impossible' happened:
   main(): signal was supposed to be fatal

host stacktrace:
==1==    at 0x5804284A: ??? (in /usr/libexec/valgrind/memcheck-amd64-linux)
==1==    by 0x58042977: ??? (in /usr/libexec/valgrind/memcheck-amd64-linux)
==1==    by 0x58042BE0: ??? (in /usr/libexec/valgrind/memcheck-amd64-linux)
==1==    by 0x58042C10: ??? (in /usr/libexec/valgrind/memcheck-amd64-linux)
==1==    by 0x580B1976: ??? (in /usr/libexec/valgrind/memcheck-amd64-linux)
==1==    by 0x580E4073: ??? (in /usr/libexec/valgrind/memcheck-amd64-linux)

sched status:
  running_tid=1


Note: see also the FAQ in the source distribution.
It contains workarounds to several common problems.
In particular, if Valgrind aborted or crashed after
identifying problems in your program, there's a good chance
that fixing those problems will prevent Valgrind aborting or
```

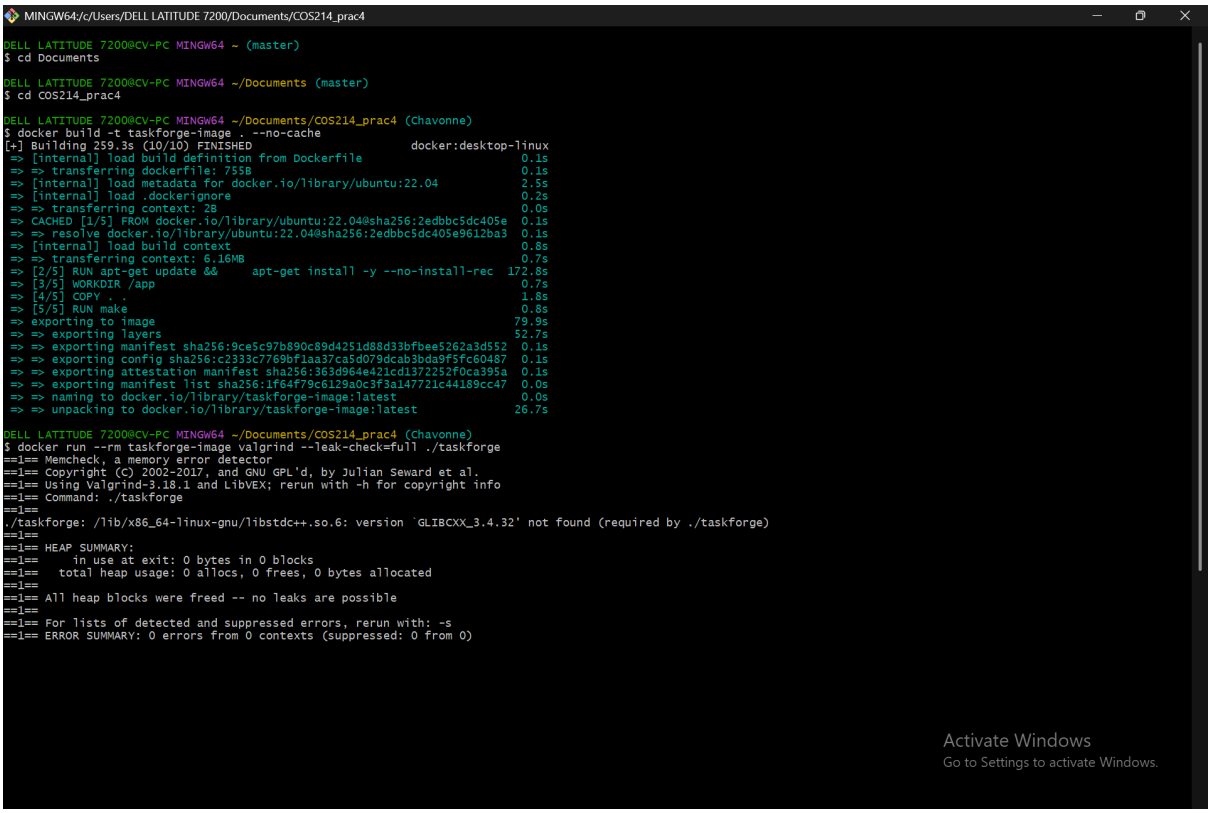
Debugging proof:

```
(gdb) break main
Breakpoint 1 at 0x62e8: file main.cpp, line 12.
(gdb) break Patient::setState
Breakpoint 2 at 0x4e1f: file Patient.cpp, line 25.
(gdb) break Patient::print
Breakpoint 3 at 0x4c4f: file Patient.cpp, line 15.
(gdb) break Patient::readmit
Breakpoint 4 at 0x5053: file Patient.cpp, line 47.
(gdb) run
Starting program: /app/taskforge
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at main.cpp:12
12      int main() {
(gdb) continue
Continuing.

Breakpoint 2, Patient::setState (this=0x555555575eb0, state=0x555555575f60) at Patient.cpp:25
25      void Patient::setState(State* state) {
(gdb) continue
Continuing.
Patient Buggy state changed to Under Examination

Breakpoint 2, Patient::setState (this=0x555555575eb0, state=0x555555575f30) at Patient.cpp:25
25      void Patient::setState(State* state) {
(gdb) continue
Continuing.
Patient Buggy state changed to In Treatment

Breakpoint 2, Patient::setState (this=0x555555575eb0, state=0x555555575f60) at Patie
nt.cpp:25
25      void Patient::setState(State* state) {
(gdb) continue
Continuing.
Patient Buggy state changed to Discharged

Breakpoint 4, Patient::readmit (this=0x555555575eb0) at Patient.cpp:47
47      void Patient::readmit() {
(gdb) step
48              if(getStateName() == "Discharged") {
(gdb) next
49                      setState(nullptr);
(gdb) next

Breakpoint 2, Patient::setState (this=0x555555575eb0, state=0x0) at Patient.cpp:25
25      void Patient::setState(State* state) {
(gdb) step
26              if(state!=nullptr){
(gdb) print state
$1 = (State *) 0x0
(gdb) next
29              this->state = state;
(gdb) next
30              cout<<"Patient "<<name<<" state changed to "<<state->getStateName()<<endl;
(gdb) next

Program received signal SIGSEGV, Segmentation fault.
0x0000555555558ea8 in Patient::setState (this=0x555555575eb0, state=0x0) at Patient.cpp:30
30              cout<<"Patient "<<name<<" state changed to "<<state->getStateName()<<endl;
(gdb) backtrace
#0  0x0000555555558ea8 in Patient::setState (this=0x555555575eb0, state=0x0)
    at Patient.cpp:30
#1  0x00005555555590ae in Patient::readmit (this=0x555555575eb0) at Patient.cpp:49
#2  0x000055555555a448 in main () at main.cpp:18
(gdb) frame 0
#0  0x0000555555558ea8 in Patient::setState (this=0x555555575eb0, state=0x0)
    at Patient.cpp:30
30              cout<<"Patient "<<name<<" state changed to "<<state->getStateName()<<endl;
(gdb) info locals
No locals.
(gdb) print state
$2 = (State *) 0x0
(gdb) quit
A debugging session is active.

        Inferior 1 [process 16] will be killed.
```

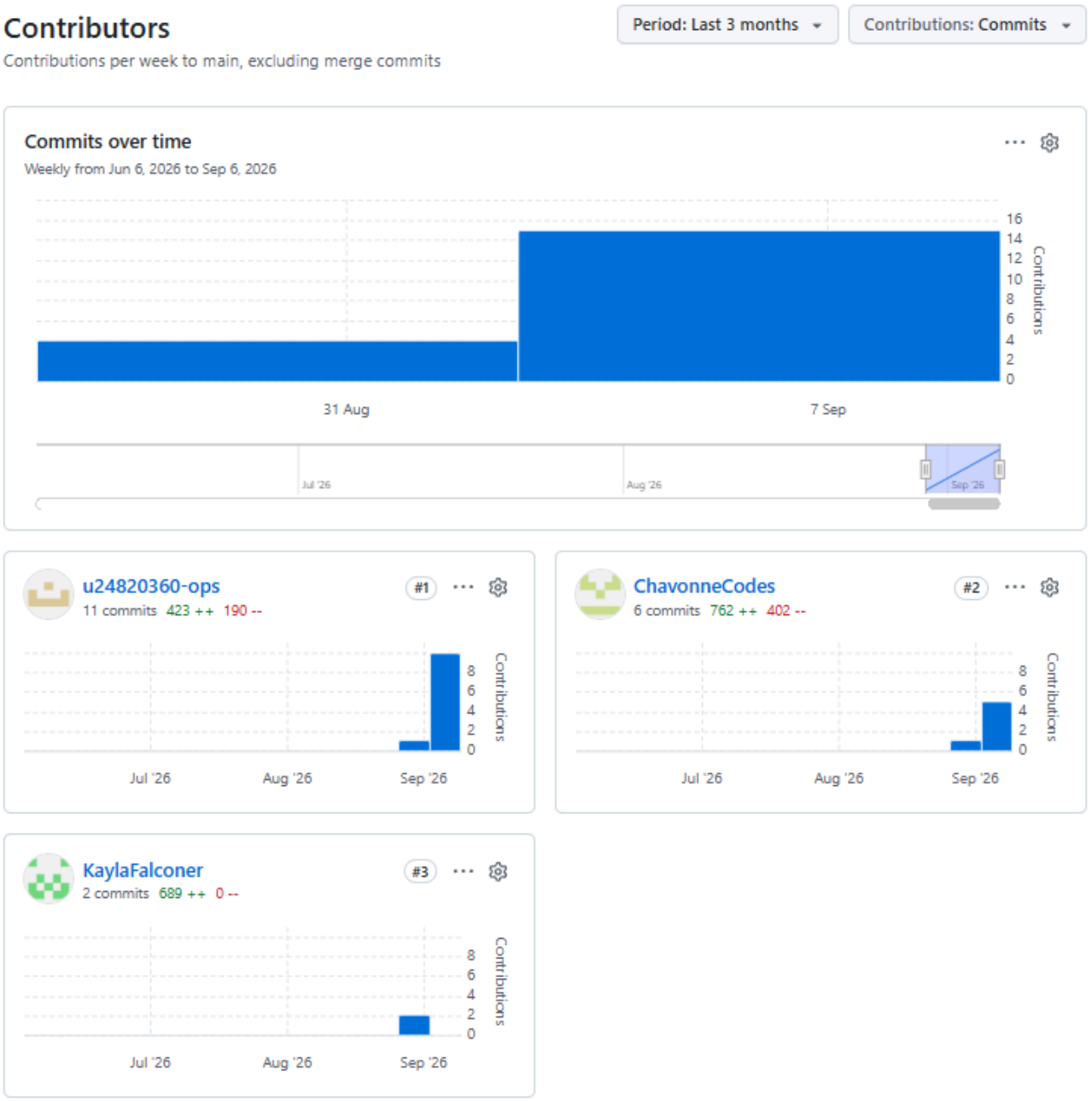
Fixed proof:

```
DELL LATITUDE 7200@CV-PC MINGW64 ~/Documents/COS214_prac4 (Chavonne)
$ docker run --rm taskforge-image
Patient Buggy state changed to Under Examination
Patient Buggy state changed to In Treatment
Patient Buggy state changed to Discharged
Patient Buggy state changed to Admitted
Patient Buggy has been readmitted.
Patient Name: Buggy (ID: BUG001, Type: Dog, Age: 2)
Current state: Admitted
-----------TESTING VET SYSTEM-------------

Lucky in isolation:
        Patient Name: Lucky (ID: 014, Type: Felis catus -> Cat, Age: 3)
Current state: Admitted
Isolation Reason: -- Terminally ill and possibly contagious. Rabies as well. --


Buddy in Emergency, Under continuous monitoring & in Isolation:
        Patient Name: Buddy (ID: 012, Type: Psittacus -> Parrot, Age: 5)
Current state: Admitted
Isolation Reason: -- Bird Flu spread being contained. --
Continuous Monitoring: -- 4 --
Emergency Priority Level: -- 3 --


Oreo in emergency:
        Patient Name: Oreo (ID: 010, Type: Canis lupus familiaris -> Dog, Age: 4)
Current state: Admitted
Emergency Priority Level: -- 6 --

--- Print All (Whole Tree Structure) ---
Unit: -- Veterinary Hospital --
Unit: -- Emergency Department --
Unit: -- Treatment Room 1 --
Patient Name: Oreo (ID: 010, Type: Canis lupus familiaris -> Dog, Age: 4)
Current state: Admitted
Emergency Priority Level: -- 6 --
Patient Name: Max (ID: 011, Type: Felis catus -> Cat, Age: 2)
Current state: Admitted
Unit: -- Treatment Room 2 --
Patient Name: Buddy (ID: 012, Type: Psittacus -> Parrot, Age: 5)
Current state: Admitted
Isolation Reason: -- Bird Flu spread being contained. --
Continuous Monitoring: -- 4 --
Emergency Priority Level: -- 3 --
Unit: -- Surgery Department --
Unit: -- Theatre 1 --
Patient Name: Milo (ID: 013, Type: Canis lupus familiaris -> Dog, Age: 1)
Current state: Admitted
Unit: -- General Care --
Unit: -- Ward A --
Patient Name: Lucky (ID: 014, Type: Felis catus -> Cat, Age: 3)
Current state: Admitted
Isolation Reason: -- Terminally ill and possibly contagious. Rabies as well. --

--- Show All Patients ---
Patient Name: Oreo (ID: 010, Type: Canis lupus familiaris -> Dog, Age: 4)
Current state: Admitted
Emergency Priority Level: -- 6 --
Patient Name: Max (ID: 011, Type: Felis catus -> Cat, Age: 2)
Current state: Admitted
Patient Name: Buddy (ID: 012, Type: Psittacus -> Parrot, Age: 5)
Current state: Admitted
Isolation Reason: -- Bird Flu spread being contained. --
Continuous Monitoring: -- 4 --
Emergency Priority Level: -- 3 --
Patient Name: Milo (ID: 013, Type: Canis lupus familiaris -> Dog, Age: 1)
Current state: Admitted
Patient Name: Lucky (ID: 014, Type: Felis catus -> Cat, Age: 3)
Current state: Admitted
Isolation Reason: -- Terminally ill and possibly contagious. Rabies as well. --

--- Show Emergency Priority Patients ---
Patient Name: Oreo (ID: 010, Type: Canis lupus familiaris -> Dog, Age: 4)
Current state: Admitted
Emergency Priority Level: -- 6 --
```

Valgrind evidence:



GitHub Repository:



Team Contribution Statement:

The tasks for this practical were split evenly among the three group members to ensure fair but collaborative participation.

Azolile Mbanga was tasked with the coding implementation of the Composite and Decorator patterns involved in Task 2, generating one runtime scenario as per Task 3, developing one UML Activity Diagram modelling the Patient lifecycle for Task 4, and providing a Makefile to test the code as per Task 6.

Chavonne Makurira was tasked with the coding implementation of the State and Iterator patterns involved in Task 2, generating one runtime scenario as per Task 3, developing one UML Activity Diagram modelling the use of the EmergencyIterator for Task 4, debugging using GDB and Valgrind as per Task 5, and creating the Dockerfile as per Task 6.

Kayla Falconer was tasked with the creation of the UML Class Diagram involved in Task 1, providing one UML Object Diagram and one UML State Diagram for Task 4, developing one UML Activity Diagram modelling the use of the FullIterator for Task 4, creating the GitHub repository as per Task 6, and compiling the PDF for the project overview.